

CSCE 614 COMPUTER ARCHITECTURE · MODULE 03 · LECTURE NOTES

Pipelining and Hazards

Overlapping instructions, and the three things that stop you.

MODULE OUTCOMES

Explain the classic five-stage pipeline and why pipelining raises throughput, not latency.

Identify structural, data, and control hazards.

Explain forwarding and when it is insufficient.

Explain why deeper pipelines increase misprediction cost.

Relate pipeline behaviour to measurable CPI.

1 Why pipeline

Executing an instruction takes several distinct steps, each using different hardware. Doing them strictly one instruction at a time leaves most of the processor idle most of the time.

Stage	What happens	Hardware used
IF — instruction fetch	Read the instruction at the program counter	Instruction cache
ID — decode	Interpret the instruction; read source registers	Decoder, register file
EX — execute	Perform arithmetic; compute memory addresses; resolve branches	ALU
MEM — memory	Perform a load or store	Data cache
WB — write back	Write the result into a register	Register file

Overlap them. While instruction 1 is in MEM, instruction 2 can be in EX, instruction 3 in ID, and instruction 4 in IF. All five units are busy and the processor completes one instruction per cycle instead of one every five.

THROUGHPUT, NOT LATENCY

Pipelining makes no individual instruction faster. Each still passes through every stage, and the pipeline registers between stages add a little delay, so per-instruction latency actually increases slightly.

What improves is the *rate* of completion. The ideal is one instruction per cycle regardless of how many stages there are — throughput equal to the stage count, latency unchanged.

This is the same trade the graphics pipeline makes in CSCE 641 Module 01, for the same reason, and it recurs throughout computer architecture.

2 Hazards

A **hazard** is anything preventing the next instruction from beginning on schedule. There are exactly three kinds, and every mechanism in the rest of this course addresses one of them.

2.1 Structural hazards

Two instructions need the same hardware in the same cycle. If there is a single memory port, an instruction fetch in IF and a load in MEM collide. The solution is duplication: separate instruction and data caches, multiple ALUs, multiple register-file ports. Structural hazards are largely engineered away in modern designs — silicon is cheap enough to duplicate what matters.

2.2 Data hazards

An instruction needs a value that an earlier, still-executing instruction has not yet produced.

```
add  r1, r2, r3    ; r1 computed in EX, written in WB
sub  r4, r1, r5    ; needs r1 in its own EX, one cycle later
```

Naively: sub stalls until add reaches WB -> three wasted cycles.

Forwarding (also called bypassing) fixes this: the value exists at the ALU output as soon as EX completes, so route it directly to the next instruction's ALU input rather than waiting for it to travel through MEM and WB. Most data hazards vanish at the cost of some wiring and multiplexers.

THE LOAD-USE HAZARD: THE ONE FORWARDING CANNOT FIX

ld r1, 0(r2) followed immediately by add r4, r1, r5.

The loaded value is not available until the end of MEM, but the add needs it in EX — which is *earlier*. Forwarding cannot send a value backwards in time, so at least one stall is unavoidable.

This is why compilers schedule an independent instruction into the slot after a load, and why issuing loads well before their results are needed is a real optimisation. On modern out-of-order cores the hardware hides much of this (Module 08), but the principle persists.

2.3 True and false dependences

Type	Pattern	Nature
RAW (read after write)	B reads what A wrote	True dependence. Real information flow. Cannot be removed by any means — the value genuinely must be computed first.
WAR (write after read)	B overwrites what A reads	False. Exists only because both instructions use the same register <i>name</i> .
WAW (write after write)	B overwrites what A wrote	False. Same cause.

The distinction matters a great deal. False dependences arise purely because an ISA offers finitely many register names, so unrelated computations are forced to reuse them. Supply more physical registers than architectural ones and rename on the fly, and the false dependences disappear entirely. That is **register renaming**, and Module 08 shows it is what makes out-of-order execution possible.

2.4 Control hazards

After a branch, which instruction should be fetched next? The answer is unknown until the branch condition is evaluated in EX. Stalling until then costs several cycles on every branch, and roughly one instruction in five is a branch.

The resolution is to guess and proceed speculatively, discarding the work if the guess was wrong. Module 04 is entirely about making that guess well.

3 Pipeline depth

Dividing the work into more, smaller stages shortens the critical path through each stage, which permits a higher clock frequency. This drove designs deeper through the 1990s.

WHY IT STOPPED

The misprediction penalty is roughly the pipeline depth: everything fetched after a mispredicted branch must be discarded. A five-stage pipeline loses about three cycles; a twenty-stage pipeline loses fifteen to twenty.

The Pentium 4 took this to 31 stages in pursuit of headline clock speed. It achieved high frequencies, poor performance per cycle, and power consumption that proved unmanageable. Intel abandoned the architecture and returned to a shorter, wider design.

Modern cores sit at 14–20 stages, which is roughly where the frequency gain and the misprediction cost balance. It is a good example of a proxy metric — clock speed — being optimised at the expense of the thing it was supposed to stand for.

4 What hazards cost, quantitatively

$$\text{CPI} = 1 + \text{stalls per instruction}$$

An ideal pipeline achieves CPI 1. Every stall adds to it, and the arithmetic is worth doing because the result is counterintuitive.

Source	Calculation	CPI contribution
Branch mispredictions	20% branches × 5% mispredicted × 15 cycles	0.15
L1 misses served by L2	30% memory ops × 5% miss × 12 cycles	0.18
Misses to main memory	30% memory ops × 2% miss × 250 cycles	1.50
Load-use stalls	10% of instructions × 1 cycle	0.10

WHY THIS COURSE SPENDS THREE MODULES ON MEMORY AND ONE ON BRANCHES

Compare the rows. A 5% branch misprediction rate contributes 0.15 to CPI. A 2% main-memory miss rate contributes 1.50 — ten times more, from a far lower event rate.

Memory is the dominant term in almost every real program's CPI, by a wide margin. That is why Modules 05 through 07 are about the memory hierarchy, and why 'is this memory-bound?' is the first question to ask of any slow code.

It is also why the RAM model's assumption of uniform memory cost is not a small inaccuracy but the single largest error in the model.

5 Consequences for code you write

- **Dependency chains limit throughput.** A sequence of dependent operations proceeds at one per *latency*, not one per cycle. Summing an array into a single accumulator is limited by addition latency; using four accumulators and combining at the end is roughly four times faster. This is exactly what the compiler did in Module 02.
- **Issue loads early.** The further ahead of its use a load is issued, the more latency can be hidden behind other work.
- **Check memory before control flow.** Branch restructuring is visible and satisfying; it is usually not where the time is. Measure IPC and cache miss rate first.
- **Let the compiler schedule.** Modern compilers schedule instructions well. The useful contribution is usually to stop blocking them — avoid pointer aliasing, keep loop bodies simple, make functions inlinable.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Onur Mutlu — Pipelining, Pipeline Hazards lectures	The full treatment including forwarding paths and the depth trade-off.
MIT 6.004 — Pipelined Processors	Builds a pipelined processor stage by stage. The best way to make forwarding concrete.
Nand2Tetris Projects 1–6	Build a CPU from gates. Not pipelined, but the understanding transfers.
Agner Fog — The microarchitecture of Intel, AMD and VIA CPUs	What real pipelines look like, core by core. Dense, and the definitive free reference.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write a loop summing an array into one accumulator, then into two, four, and eight. Measure all four and explain the curve in terms of dependency chains and addition latency. Find where it stops improving and say why.
- 2 Construct a load-use hazard in C and inspect the generated assembly. Does the compiler schedule an independent instruction into the gap?

- 3 Measure a loop with a long dependency chain against one with independent operations, matched for instruction count. Report IPC for both using perf stat.
- 4 Compute the CPI contribution of branches and of memory for a program of your own, using hardware counters. Which dominates?
- 5 Write two versions of a loop that differ only in whether consecutive iterations are dependent. Predict the ratio before measuring, then measure.
- 6 Look up the pipeline depth of your own processor and compute the expected misprediction penalty. Compare with the value you will measure in Module 04.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Does pipelining reduce the latency of an instruction? What does it improve, and by how much ideally?
- 2 Name the three hazard classes with a one-line cause for each.
- 3 What does forwarding do, and give the specific case it cannot fix.
- 4 Why are WAR and WAW called false dependences, and what removes them?
- 5 Why did very deep pipelines fall out of favour?
- 6 Compute the CPI contribution of a 2% miss rate to 250-cycle memory, with 30% of instructions accessing memory. Compare it with a 5% branch misprediction rate at 15 cycles.